Lecture08: Date and Functions in JS

Date

In-Depth Lecture Notes: The JavaScript Date Object

1. The Core Concept: A Single Moment in Time

Before we look at any code, we must understand the "first thought" principle of JavaScript dates.

A Date object is not a calendar; it's a single, massive number. This number represents the total milliseconds that have passed since a universal starting point: the **Unix Epoch**, which is midnight on **January 1st, 1970, UTC** (01-jan-1970).

Every date you create is just a wrapper around this timestamp.

This makes it easy to compare dates and calculate durations.

2. Creating Date Objects

You create a Date object using the new Date() constructor.

A. Create a Date for the Current Moment
If you provide no arguments, you get the current date and time from the user's device.

```
// Creates a Date object for right now.
const now = new Date();
console.log(now); // e.g., Mon Oct 28 2024 03:30:00 GMT+0530 (India Standard Time)
```

B. Create a Date from a Timestamp
You can create a date from that big "milliseconds" number we talked about. This is how servers and databases often communicate time.

```
// Date.now() is a quick way to get the CURRENT timestamp as a number.
const currentTimestamp = Date.now(); // e.g., 1759262295036
```

```
console.log(currentTimestamp);

// Now, let's create a Date object from that number.
const da = new Date(1759262295036);
console.log(da); // This will show the date and time corresponding to that timestamp.
```

C. Create a Specific Date (The Recommended Way)
This is the most reliable way to create a specific date. You provide the components as numbers in this order: year, month, day, hours, minutes, seconds, ms.

```
// new Date(year, month, day, hours, minutes, seconds, ms)
const myDate = new Date(2025, 8, 4, 6, 20, 11, 125);
console.log(myDate.toString()); // Thu Sep 04 2025 06:20:11 GMT+0530 (India Standard Time)
```

CRITICAL GOTCHA: Months are Zero-Indexed!
This is the most common source of bugs with JavaScript dates.

0 = January

1 = February

...

8 = **September** (as in the example above)

11 = December

3. Getting Information from a Date Object (Getters)

Once you have a Date object, you can extract its individual parts. These methods return values based on the user's local timezone.

```
const now = new Date(); // Let's pretend it's Sep 4, 2025, a Thursday

console.log(now.getFullYear()); // 2025 (The full 4-digit year)
console.log(now.getMonth());    // 8 (Remember, 0-indexed, so 8 is September)
```

```
console.log(now.getDate());     // 4 (The day of the month, 1-31)
console.log(now.getDay());      // 4 (The day of the week: 0=Sunday, 1=Monday, ..., 4=Thursday)
console.log(now.getHours());    // e.g., 6 (The hour, 0-23)
console.log(now.getMinutes());  // e.g., 20 (The minute, 0-59)
```

4. Changing a Date: Setters and Mutability

Date objects are **mutable**, meaning you can change their value in place using "setter" methods.

```
const da = new Date(1759262295036); // Creates a specific date
console.log("Before:", da);

// Let's change the month to May (index 4)
da.setMonth(4);

console.log("After:", da); // The ORIGINAL 'da' object has been changed.
```

This is different from primitives like strings or numbers. Calling a setter method **mutates** the original object.

5. Date Auto-Correction (Rollover)

The Date object is smart about invalid dates. If you give it a value that's out of range, it will "roll over" to the next logical date.

Your example new Date(2025, 1, 30) is a perfect demonstration. You are asking for **February 30th, 2025**.

```
// 2025 is not a leap year, so February has 28 days.
// You are asking for Feb 28 + 2 days.
const a = new Date(2025, 1, 30);

// The date object auto-corrects to March 2nd, 2025.
console.log(a); // Outputs: Sun Mar 02 2025 ...
```

This feature can be useful for date math (like adding 30 days to a date), but it can also hide bugs if you aren't expecting it.

6. Formatting Dates for Display

A raw Date object isn't very user-friendly. You must format it into a string for display.

```
const now = new Date();

// .toString() is the default, verbose format in the local timezone.
console.log(now.toString());
// e.g., Mon Oct 28 2024 04:15:00 GMT+0530 (India Standard Time)

// .toDateString() gives you just the date part.
console.log(now.toDateString());
// e.g., Mon Oct 28 2024

// .toISOString() gives the universal, unambiguous UTC time. The 'Z' means UTC.
// This is what you send to servers.
console.log(now.toISOString());
// e.g., 2024-10-27T22:45:00.000Z

// .toLocaleString() formats the date and time in a way that is natural
// for the user's region and language. BEST for user display.
console.log(now.toLocaleString());
// e.g., in India: 28/10/2024, 4:15:00 am
// e.g., in the US: 10/27/2024, 4:15:00 PM
```

Summary of Key "Gotchas"

Months are 0-indexed. This is the #1 source of bugs.

Dates are Mutable. Setter methods change the original object.

String Parsing is Unreliable. Never use new Date("some-date-string").

Timezones are Complex. Always be aware of whether you are working with local time or UTC time. Send UTC timestamps to servers.

/ console.log(now.toString());
// console.log(now.toDateString());
// console.log(now.toISOString());
// console.log(now.toLocaleString());

The JavaScript Date object is **notorious** for being one of the most criticized parts of JavaScript. Here's why:

Major Problems:

1. Months start at 0, but days start at 1

```
new Date(2025, 0, 1);  // January 1, 2025 (month is 0-indexed)
new Date(2025, 11, 25); // December 25, 2025 (11 = December!)
// This is confusing and error-prone!
```

2. Mutable (can be changed accidentally)

```
const date = new Date(2025, 9, 1);
date.setMonth(10); // Oops! Date is modified
console.log(date); // Changed to November!
// This causes bugs in apps
```

3. Weird quirks and bugs

```
new Date(2025, 1, 30); // Feb 30? Becomes March 2!
new Date('2025-02-30'); // Invalid Date

// Inconsistent behavior
new Date(2025, 0); // Jan 1, 2025 00:00:00
new Date(2025);    // Jan 1, 1970 plus 2025 milliseconds! 😱
```

The Solution:

Because Date is so bad, developers use libraries:

Day.js (lightweight, 2KB)

date-fns (functional, tree-shakeable)

Luxon (modern, powerful)

Functions

Part 1: The Core Concept - What is a Function?

First Thought: A function is a reusable "code machine" or a "recipe."

Imagine you have a task you need to do over and over, like making a sandwich. Instead of re-writing the instructions (get bread, get cheese, etc.) every single time, you write the recipe down once and give it a name, like "Make Turkey Sandwich."

The **recipe** is the **function**.

The **ingredients** you give it (like "turkey" or "ham") are the **parameters**.

The finished sandwich is the **return value**.

Why do we need them? The DRY Principle: Don't **R**epeat **Y**ourself. Functions let you write a piece of logic once and reuse it hundreds of times.

A. The Three Core Parts

Declaration (Writing the Recipe): This is where you define the function and give it a name.

Calling (Using the Recipe): This is where you execute the function.

Parameters & Return Value (Ingredients & Finished Dish): How you pass data *into* a function and get a result *out of* it.

B. Basic Syntax

```
// 1. DECLARATION: Writing the recipe
//    - `function` is the keyword.
//    - `greet` is the name of our function.
//    - `(name)` is the parameter, a placeholder for the input.
function greet(name) {
```

```
  // The code inside the {} is the "body" of the function.
  const message = `Hello, ${name}!`;
  return message; // 3. RETURN VALUE: Giving back the result.
}

// 2. CALLING: Using the recipe with a specific ingredient ("Alice")
const greetingForAlice = greet("Alice");
console.log(greetingForAlice); // Outputs: "Hello, Alice!"

const greetingForBob = greet("Bob");
console.log(greetingForBob);   // Outputs: "Hello, Bob!"
```

Parameter: The variable name inside the function's parentheses (name). It's a placeholder.

Argument: The actual value you pass into the function when you call it ("Alice").

Return Value: The value a function sends back using the return keyword. If a function has no return statement, it automatically returns undefined.

Part 2: Different Ways to Create Functions

There are three main ways to create functions in JavaScript, and their differences are important.

A. Function Declaration

This is the classic way we saw above.

```
function add(a, b) {
  return a + b;
}
```

Key Feature: Hoisting. Function declarations are "hoisted," meaning the JavaScript engine "lifts" them to the top of their scope before the code is executed. This means you can call a function *before* you define it in the code.

```
console.log(add(2, 3)); // 5 (This works!)
```

```
function add(a, b) {
  return a + b;
}
```

B. Function Expression

Here, you create an anonymous (unnamed) function and assign it to a variable.

```
const subtract = function(a, b) {
  return a - b;
};
```

Key Feature: Not Hoisted. A function expression is not hoisted. It behaves just like a variable. You cannot call it before it is defined.

```
// subtract(10, 5); // This would throw a ReferenceError!

const subtract = function(a, b) {
  return a - b;
};

console.log(subtract(10, 5)); // 5 (This works)
```

C. Arrow Functions (ES6 - The Modern Way)

First Thought: "A shorter, cleaner syntax for writing functions."

Arrow functions are a more concise way to write function expressions. They are extremely popular in modern JavaScript.

```
// A function expression
const multiply = function(a, b) {
  return a * b;
}

// The same function as an arrow function
const multiplyArrow = (a, b) ⇒ {
```

```
  return a * b;
}
```

Syntax Rules for Arrow Functions:

Implicit Return: If the function body is just a single expression, you can remove the curly braces {} and the return keyword.

const multiplyShort = (a, b) ⇒ a * b; // The result of a * b is automatically returned.

Single Parameter: If there is only one parameter, you can remove the parentheses ().

const square = x ⇒ x * x;

You've already seen this in action with the array sort method: (a, b) ⇒ a - b. This is a concise arrow function that implicitly returns the result of a - b.

Part 3: Advanced Parameter Concepts

A. Default Parameters (ES6)

You can provide a default value for a parameter in case it's not passed in (i.e., it's undefined).

```
// The default value for 'name' is "Guest".
function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}

greet("Alice"); // Hello, Alice!
greet();        // Hello, Guest!
```

B. Rest Parameters (...) (ES6)

This allows a function to accept an **indefinite number of arguments** and gather them into a **true array**.

First Thought: "Put the rest of the ingredients into a single bag."

```
// The '...numbers' gathers all arguments passed into an array called 'numbers'.
function sumAll(...numbers) {
  let total = 0;
  for (const num of numbers) {
    total += num;
  }
  return total;
}

console.log(sumAll(1, 2));          // 3
console.log(sumAll(1, 2, 3, 4, 5)); // 15
console.log(sumAll(10));            // 10
```

The rest parameter must be the **last** parameter in the function definition.

Part 4: Scope - Where Do Variables Live?

First Thought: "A function is its own little world."

Global Scope: Variables declared outside of any function are "global" and can be accessed from anywhere.

Function Scope (Local Scope): Variables declared inside a function (with let, const, or var) **only exist inside that function**. They cannot be accessed from the outside.

```
let globalVar = "I am global";

function myFunction() {
  let localVar = "I am local";
  console.log(globalVar); // "I am global" (Can access global variables)
  console.log(localVar);  // "I am local"
}

myFunction();

// console.log(localVar); // ReferenceError: localVar is not defined
```

This is a critical feature that prevents variables from different parts of your code from interfering with each other.

Rest vs Spread: The Same Syntax, Opposite Purposes

Here's the mind-bending part: **they use the exact same syntax (...) but do opposite things!**

The Key Difference

Spread = **Unpack/Expand** (takes one thing and breaks it apart)

Rest = **Pack/Collect** (takes many things and combines them into one)

Think of it like a suitcase:

Spread = Opening a suitcase and spreading clothes everywhere

Rest = Gathering scattered clothes and packing them into a suitcase

SPREAD Operator (Expanding/Unpacking)

Takes an array or object and **spreads its elements out**.

Arrays

```
const numbers = [1, 2, 3];

// Spread: Unpack the array into individual elements
console.log(...numbers);  // Output: 1 2 3 (three separate values)
console.log(numbers);     // Output: [1, 2, 3] (one array)

// Practical use: Combining arrays
const moreNumbers = [4, 5, 6];
const combined = [...numbers, ...moreNumbers];
console.log(combined);  // [1, 2, 3, 4, 5, 6]

// Practical use: Copying an array
const copy = [...numbers];

// Practical use: Passing array elements as function arguments
function add(a, b, c) {
```

```
    return a + b + c;
}
console.log(add(...numbers));  // Same as add(1, 2, 3)
```

Objects

```
const person = { name: "Alice", age: 25 };

// Spread: Unpack object properties
const updatedPerson = { ...person, city: "NYC" };
console.log(updatedPerson);
// { name: "Alice", age: 25, city: "NYC" }

// Overwriting properties
const olderPerson = { ...person, age: 30 };
// { name: "Alice", age: 30 }
```

REST Operator (Collecting/Packing)

Takes **multiple individual elements** and **collects them into an array**.

In Function Parameters

```
// Rest: Collect all remaining arguments into an array
function sum(...numbers) {
    console.log(numbers);  // numbers is an array
    return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(1, 2, 3, 4, 5));  // 15
// Inside the function, numbers = [1, 2, 3, 4, 5]

// Mixing regular params with rest
function introduce(greeting, ...names) {
    console.log(greeting);  // "Hello"
    console.log(names);     // ["Alice", "Bob", "Charlie"]
}
```

```
introduce("Hello", "Alice", "Bob", "Charlie");
```

In Destructuring (Arrays)

```
const numbers = [1, 2, 3, 4, 5];

// Rest: Collect the "rest" of the elements
const [first, second, ...others] = numbers;

console.log(first);   // 1
console.log(second);  // 2
console.log(others);  // [3, 4, 5] - collected into an array!
```

In Destructuring (Objects)

```
const person = {
    name: "Alice",
    age: 25,
    city: "NYC",
    country: "USA"
};

// Rest: Collect remaining properties
const { name, ...otherInfo } = person;

console.log(name);       // "Alice"
console.log(otherInfo);  // { age: 25, city: "NYC", country: "USA" }
```

Side-by-Side Comparison

```
// SPREAD: Taking an array and spreading it out
const arr = [1, 2, 3];
console.log(...arr);  // 1 2 3 (three separate values)

// REST: Taking separate values and collecting them
```

```
function example(...params) {
   console.log(params);  // [1, 2, 3] (one array)
}
example(1, 2, 3);
```

How to Remember Which is Which

Look at WHERE it's used:

RIGHT side of assignment = SPREAD (expanding)

```
const copy = [...original];  // Spreading original
```

LEFT side of assignment = REST (collecting)

```
const [first, ...rest] = array;  // Collecting rest
```

Function call = SPREAD (unpacking arguments)

```
myFunction(...array);  // Spreading array into arguments
```

Function definition = REST (collecting parameters)

```
function myFunction(...params) { }  // Collecting params
```

Real-World Example: Both Together!

```
function logDetails(name, age, ...hobbies) {
   // REST: hobbies collects remaining arguments into array
   console.log(`${name} is ${age} years old`);
   console.log("Hobbies:", hobbies);
}

const person = ["Alice", 25, "reading", "coding", "hiking"];

// SPREAD: Unpack array into individual arguments
logDetails(...person);

// Output:
```

```
// Alice is 25 years old
// Hobbies: ["reading", "coding", "hiking"]
```

Quick Rule

Spread = ... takes ONE thing → makes it MANY

Rest = ... takes MANY things → makes it ONE

Arrow Function In-depth

In-Depth Guide: Arrow Functions (⇒)

1. The Core Purpose: A Better function

First Thought: Arrow functions were created to solve two major problems with traditional function expressions:

They were too verbose for simple, one-line operations (like in array methods).

Their behavior with the this keyword was a constant source of confusion and bugs.

An arrow function is a **syntactically compact alternative to a regular function expression**.

2. Syntax Breakdown: From Verbose to Concise

Let's transform a traditional function expression into its shortest possible arrow function form.

Start: Traditional Function Expression

```
const add = function(a, b) {
  return a + b;
};
```

Step 1: Replace function with ⇒
Remove the function keyword and place a "fat arrow" ⇒ after the parameters.

```
const add = (a, b) ⇒ {
  return a + b;
};
```

Step 2: Implicit Return
This is the biggest syntax win. If the function body consists of only **a single return expression**, you can remove the curly braces {} and the return keyword. The result of the expression is returned automatically.

```
const add = (a, b) ⇒ a + b;
```

Step 3: Parentheses for a Single Parameter
If the function has **exactly one parameter**, you can also remove the parentheses around it.

```
// Traditional
const square = function(x) {
  return x * x;
};

// Arrow function with one parameter
const squareArrow = x ⇒ x * x;
```

If you have zero or more than one parameter, the parentheses are **required**.

```
const sayHello = () ⇒ "Hello"; // Zero parameters
const sum = (a, b, c) ⇒ a + b + c; // Three parameters
```

Important Gotcha: Returning an Object Literal
If you want to implicitly return an object literal, you must wrap it in parentheses to distinguish it from a function body's curly braces.

```
// WRONG: This is interpreted as a function body with a label, returning undefined.
const createUser = name ⇒ { name: name };

// CORRECT: Wrap the object in parentheses.
const createUserCorrect = name ⇒ ({ name: name });
```

Callback Function

A callback function in JavaScript is a function passed as an argument to another function. This allows the receiving function, often called a higher-order function, to execute the callback function at a specific point during its operation, typically after a particular task is completed.

That's the entire concept.

The Analogy: Ordering a Pizza

Let's break this down.

The Scenario: You want a pizza, but it's going to take a while to cook. You don't want to just stand at the counter and wait. You want to go do something else.

You place your order (You call a function).
You go to the pizza place and say, "I'd like a large pepperoni pizza." This is you calling a function, orderPizza().

You provide the callback (You give them your phone number).
The cashier says, "Great, that will be 20 minutes. What's your number so we can call you when it's ready?"
That phone number is your **callback function**. It's the instruction you are leaving behind for what to do *after* the main task (making the pizza) is complete.

Your "phone number" function might be called pickUpPizza.

The long task happens (The pizza is made).
You leave and go about your business. The pizza place is busy making your pizza. This is the asynchronous part—a task is happening in the background, and your main program isn't blocked waiting for it.

The task finishes, and they "call you back."
After 20 minutes, the pizza is ready. The cashier looks at your order, finds your phone number (pickUpPizza), and calls it.
This is the **callback execution**.

The orderPizza function is now "calling back" the pickUpPizza function that you provided earlier.

The Code Equivalent

Let's translate this directly into JavaScript code.

```
// This is your callback function. It's the "phone number" you'll leave behind.
// It describes what to do once the pizza is ready.
function pickUpPizza() {
  console.log("Pizza is ready! Driving to the store to pick it up.");
}


function orderPizza(callback) {


  console.log("Placing the pizza order...");

    console.log("Pizza is cooked!");

    callback();


}

// --- Let's run the program ---
// We call orderPizza and give it our pickUpPizza function as the callback
argument.
orderPizza(pickUpPizza);

// This line will run immediately, while the pizza is still "cooking".
console.log("I'm not waiting at the store. I'm at home, coding.");
```

```
Placing the pizza order...
I'm not waiting at the store. I'm at home, coding.
(after 2 seconds)
Pizza is cooked!
Pizza is ready! Driving to the store to pick it up.
```

Other use of Callback Function

```
function add(num1,num2){
    return num1+num2;
}
```

```
function sub(num1,num2){
    return num1-num2;
}

function mul(num1,num2){
    return num1*num2;
}


function calculator(num1,num2,operation){
    const ans = operation(num1,num2);

    console.log(ans);
}

calculator(10,2,function(x,y){
    return x/y;
});


const arr = [10,7,3,25,8,-2,-3];

arr.mySort = function(callback){

    for(let i=0;i<this.length;i++){
        for(j=0;j<this.length-1;j++){
            if(callback(this[j],this[j+1])){
                let num = this[j];
                this[j] = this[j+1];
                this[j+1] = num;
            }
        }
    }

    return this;
}
```

```
console.log(arr.mySort((a,b)⇒ a>b));
```